

mb4882-W4111-Spring-2026-HW4-non-programming-v1.ipynb

Introduction

This notebook defines and provides the implementation template for W4111 - Introduction to Databases, Spring 2026 Homework 4 for the **non-programming track**. The [Ed thread](#) will provide detailed submission instructions and also be the place where we discuss the homework, and answer clarifying questions. The homework is due on 08-May-2026 at 11:59 PM.

Homework Overview

Submission Instructions

Please see the Ed post.

Part 0 — Setup

iPython-SQL

```
In [1]: %pip install ipython-sql sqlalchemy pandas pymysql cryptography
```

```
Collecting ipython-sql
  Using cached ipython_sql-0.5.0-py3-none-any.whl.metadata (17 kB)
Collecting sqlalchemy
  Using cached sqlalchemy-2.0.49-cp310-cp310-macosx_11_0_arm64.whl.metadata (9.5 kB)
Collecting pandas
  Using cached pandas-2.3.3-cp310-cp310-macosx_11_0_arm64.whl.metadata (91 kB)
Collecting pymysql
  Using cached pymysql-1.1.3-py3-none-any.whl.metadata (4.3 kB)
Collecting cryptography
  Using cached cryptography-48.0.0-cp39-abi3-macosx_10_9_universal2.whl.metadata (4.3 kB)
Collecting prettytable (from ipython-sql)
  Using cached prettytable-3.17.0-py3-none-any.whl.metadata (34 kB)
Requirement already satisfied: ipython in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython-sql) (8.37.0)
Collecting sqlparse (from ipython-sql)
  Using cached sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: six in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython-sql) (1.17.0)
Collecting ipython-genutils (from ipython-sql)
  Using cached ipython_genutils-0.2.0-py2.py3-none-any.whl.metadata (755 bytes)
Requirement already satisfied: typing-extensions>=4.6.0 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from sqlalchemy) (4.15.0)
Collecting numpy>=1.22.4 (from pandas)
  Using cached numpy-2.2.6-cp310-cp310-macosx_14_0_arm64.whl.metadata (62 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from pandas) (2.9.0.post0)
Collecting pytz>=2020.1 (from pandas)
  Using cached pytz-2026.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Using cached tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Using cached cffi-2.0.0-cp310-cp310-macosx_11_0_arm64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Using cached pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Requirement already satisfied: decorator in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (5.2.1)
Requirement already satisfied: exceptiongroup in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (1.3.1)
Requirement already satisfied: jedi>=0.16 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (0.19.2)
Requirement already satisfied: matplotlib-inline in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (0.2.2)
Requirement already satisfied: pexpect>4.3 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (4.9.0)
```

```

Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (3.0.52)
Requirement already satisfied: pygments>=2.4.0 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (2.20.0)
Requirement already satisfied: stack_data in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (0.6.3)
Requirement already satisfied: traitlets>=5.13.0 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from ipython->ipython-sql) (5.15.0)
Requirement already satisfied: wcwidth in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython->ipython-sql) (0.7.0)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from jedi=>0.16->ipython->ipython-sql) (0.8.7)
Requirement already satisfied: ptyprocess>=0.5 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from pexpect>4.3->ipython->ipython-sql) (0.7.0)
Requirement already satisfied: executing>=1.2.0 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from stack_data->ipython->ipython-sql) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from stack_data->ipython->ipython-sql) (3.0.1)
Requirement already satisfied: pure_eval in /Users/minjae/miniconda3/envs/database_hw4/lib/python3.10/site-packages (from stack_data->ipython->ipython-sql) (0.2.3)
Using cached ipython_sql-0.5.0-py3-none-any.whl (20 kB)
Using cached sqlalchemy-2.0.49-cp310-cp310-macosx_11_0_arm64.whl (2.2 MB)
Using cached pandas-2.3.3-cp310-cp310-macosx_11_0_arm64.whl (10.8 MB)
Using cached pymysql-1.1.3-py3-none-any.whl (45 kB)
Using cached cryptography-48.0.0-cp39-abi3-macosx_10_9_universal2.whl (8.0 MB)
Using cached cffi-2.0.0-cp310-cp310-macosx_11_0_arm64.whl (180 kB)
Using cached numpy-2.2.6-cp310-cp310-macosx_14_0_arm64.whl (5.3 MB)
Using cached pytz-2026.2-py2.py3-none-any.whl (510 kB)
Using cached tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Using cached ipython_genutils-0.2.0-py2.py3-none-any.whl (26 kB)
Using cached prettytable-3.17.0-py3-none-any.whl (34 kB)
Using cached pycparser-3.0-py3-none-any.whl (48 kB)
Using cached sqlparse-0.5.5-py3-none-any.whl (46 kB)
Installing collected packages: pytz, ipython-genutils, tzdata, sqlparse, sqlalchemy, pymysql, pycparser, prettytable, numpy, pandas, cffi, ipython-sql, cryptography
13/13 [cryptography] — 9/13 [pandas]
Successfully installed cffi-2.0.0 cryptography-48.0.0 ipython-genutils-0.2.0 ipython-sql-0.5.0 numpy-2.2.6 pandas-2.3.3 prettytable-3.17.0 pycparser-3.0 pymysql-1.1.3 pytz-2026.2 sqlalchemy-2.0.49 sqlparse-0.5.5 tzdata-2026.2
Note: you may need to restart the kernel to use updated packages.

```

In [2]: `%load_ext sql`

Set the proper root user ID and password for MySQL in the python cell below.

```
In [3]: mysql_root_user = "root"
mysql_root_password = "alswo4064"
mysql_url = f"mysql+pymysql://{mysql_root_user}:{mysql_root_password}@localhost"
```

```
In [4]: from sqlalchemy import create_engine, text

engine = create_engine(mysql_url)
with engine.connect() as c:
    c.execute(text("SELECT 1"))
print("connection OK")
```

connection OK

```
In [5]: mysql_url
```

```
Out[5]: 'mysql+pymysql://root:alswo4064@localhost'
```

```
In [7]: %sql mysql+pymysql://root:alswo4064@localhost
```

```
In [9]: %config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

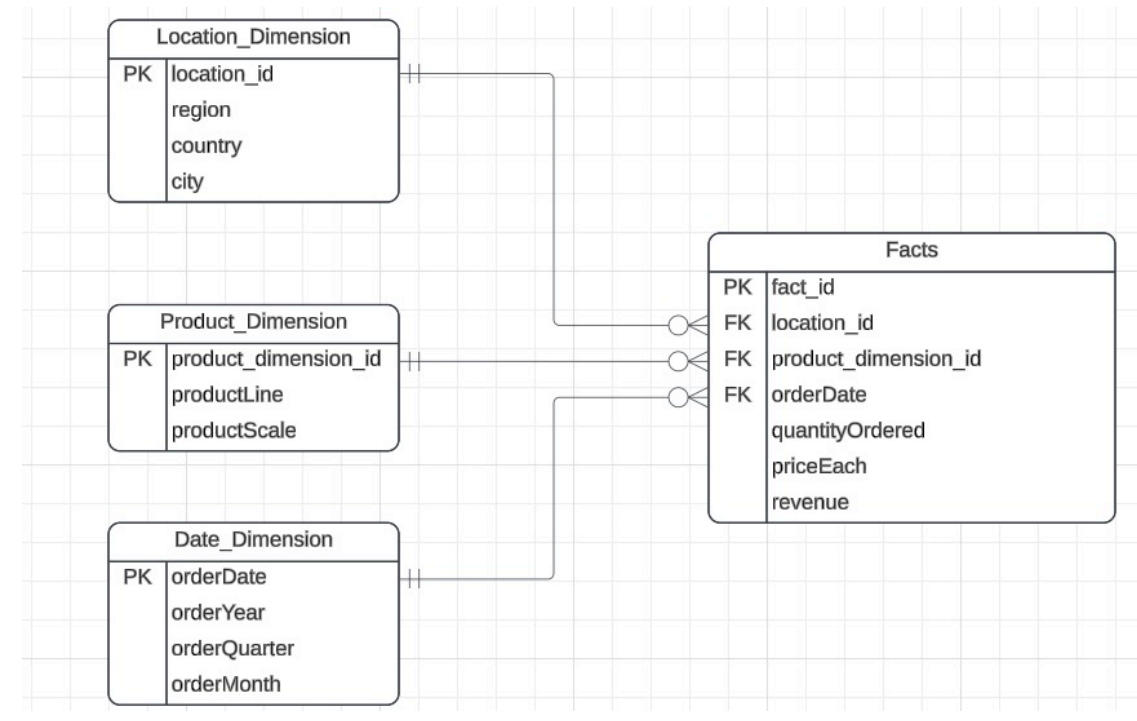
```
In [ ]: # Check whether SQL is successfully connected
%sql select * from db_book.student where dept_name="Comp. Sci."
```

* mysql+pymysql://root:***@localhost
4 rows affected.

```
Out [ ]:
```

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
54321	Williams	Comp. Sci.	54
76543	Brown	Comp. Sci.	58

Classmodels Star Schema



Classicmodels Star Schema

Note: You must first complete the **DDL** (Data Definition Language) for the schema and loading the data tasks below. The following cells show you what my tables and data look like to help you.

The preceding diagram is an overview of a star schema for the Classicmodels dataset that we have been using in the class. The following queries provide examples of the data transformed and loaded into the schema.

```
In [ ]: %sql use classicmodels_olap;

* mysql+pymysql://root:***@localhost
0 rows affected.
```

```
Out[ ]: []
```

```
In [16]: %sql select * from facts limit 10;

* mysql+pymysql://root:***@localhost
10 rows affected.
```

Out [16]:

fact_id	location_id	product_dimension_id	orderDate	quantityOrdered	priceEach	re
---------	-------------	----------------------	-----------	-----------------	-----------	----

1	53	1	2003-01-29	26	214.30	51
2	66	1	2003-03-24	29	197.16	5
3	63	1	2003-05-28	38	205.73	7
4	80	1	2003-07-24	37	186.44	68
5	60	1	2003-09-19	45	182.16	8
6	89	1	2003-10-20	21	212.16	44
7	5	1	2003-11-06	34	207.87	70
8	91	1	2003-11-13	23	180.01	47
9	19	1	2003-11-25	42	203.59	85
10	12	1	2003-12-05	47	203.59	95

In [17]: `%sql select * from location_dimension;`

* mysql+pymysql://root:***@localhost
96 rows affected.

Out[17]:

location_id	country	city	region
1	Australia	Chatswood	APAC
2	Australia	Glen Waverly	APAC
3	Australia	Melbourne	APAC
4	Australia	North Sydney	APAC
5	Australia	South Brisbane	APAC
6	Austria	Graz	EMEA
7	Austria	Salzburg	EMEA
8	Belgium	Bruxelles	EMEA
9	Belgium	Charleroi	EMEA
10	Canada	Montréal	AMER
11	Canada	Tsawassen	AMER
12	Canada	Vancouver	AMER
13	Denmark	Århus	EMEA
14	Denmark	Kobenhavn	EMEA
15	Finland	Espoo	EMEA
16	Finland	Helsinki	EMEA
17	Finland	Oulu	EMEA
18	France	Lille	EMEA
19	France	Lyon	EMEA
20	France	Marseille	EMEA
21	France	Nantes	EMEA
22	France	Paris	EMEA
23	France	Reims	EMEA
24	France	Strasbourg	EMEA
25	France	Toulouse	EMEA
26	France	Versailles	EMEA
27	Germany	Aachen	EMEA
28	Germany	Berlin	EMEA
29	Germany	Brandenburg	EMEA
30	Germany	Cunewalde	EMEA
31	Germany	Frankfurt	EMEA
32	Germany	Köln	EMEA

location_id	country	city	region
33	Germany	Leipzig	EMEA
34	Germany	Mannheim	EMEA
35	Germany	München	EMEA
36	Germany	Munich	EMEA
37	Germany	Münster	EMEA
38	Germany	Stuttgart	EMEA
39	Hong Kong	Central Hong Kong	APAC
40	Ireland	Cork	EMEA
41	Ireland	Dublin	EMEA
42	Israel	Herzlia	EMEA
43	Italy	Bergamo	EMEA
44	Italy	Milan	EMEA
45	Italy	Reggio Emilia	EMEA
46	Italy	Torino	EMEA
47	Japan	Kita-ku	APAC
48	Japan	Minato-ku	APAC
49	Netherlands	Amsterdam	EMEA
50	New Zealand	Auckland	APAC
51	New Zealand	Auckland	APAC
52	New Zealand	Wellington	APAC
53	Norway	Stavern	EMEA
54	Norway	Bergen	EMEA
55	Norway	Oslo	EMEA
56	Philippines	Makati City	APAC
57	Poland	Warszawa	EMEA
58	Portugal	Lisboa	EMEA
59	Russia	Saint Petersburg	EMEA
60	Singapore	Singapore	APAC
61	South Africa	Hatfield	EMEA
62	Spain	Barcelona	EMEA
63	Spain	Madrid	EMEA
64	Spain	Sevilla	EMEA

location_id	country	city	region
65	Sweden	Bräcke	EMEA
66	Sweden	Luleå	EMEA
67	Switzerland	Bern	EMEA
68	Switzerland	Fribourg	EMEA
69	Switzerland	Genève	EMEA
70	UK	Cowes	EMEA
71	UK	Liverpool	EMEA
72	UK	London	EMEA
73	UK	Manchester	EMEA
74	USA	Allentown	AMER
75	USA	Boston	AMER
76	USA	Brickhaven	AMER
77	USA	Bridgewater	AMER
78	USA	Brisbane	AMER
79	USA	Burbank	AMER
80	USA	Burlingame	AMER
81	USA	Cambridge	AMER
82	USA	Glendale	AMER
83	USA	Las Vegas	AMER
84	USA	Los Angeles	AMER
85	USA	Nashua	AMER
86	USA	New Bedford	AMER
87	USA	New Haven	AMER
88	USA	Newark	AMER
89	USA	NYC	AMER
90	USA	Pasadena	AMER
91	USA	Philadelphia	AMER
92	USA	San Diego	AMER
93	USA	San Francisco	AMER
94	USA	San Jose	AMER
95	USA	San Rafael	AMER
96	USA	White Plains	AMER

The **location** is the **country** and **city** of the **customer** that placed the order. I manually entered the **regions** to make the data more interesting and make the location dimension a little deeper. The regions are:

- **EMEA** is Europ, Middle East, Africa
- **APAC** is Asia and the Pacific
- **AMER** is Americas

The mapping of **country** to **region** is below.

You can write **UPDATE ... SET ... WHERE ...** statements to set the values. The countries in **APAC** are **Australia, Hong Kong, Japan, New Zealand, Philippines, Singapore**. The two countries in **AMER** are **Canada, USA**. All other countries are in **EMEA**. Manually setting the regions takes about 10 minutes.

```
In [19]: %sql select distinct country, region from location_dimension order by region
* mysql+pymysql://root:***@localhost
28 rows affected.
```

Out [19]:

country	region
Canada	AMER
USA	AMER
Australia	APAC
Hong Kong	APAC
Japan	APAC
New Zealand	APAC
Philippines	APAC
Singapore	APAC
Austria	EMEA
Belgium	EMEA
Denmark	EMEA
Finland	EMEA
France	EMEA
Germany	EMEA
Ireland	EMEA
Israel	EMEA
Italy	EMEA
Netherlands	EMEA
Norway	EMEA
Norway	EMEA
Poland	EMEA
Portugal	EMEA
Russia	EMEA
South Africa	EMEA
Spain	EMEA
Sweden	EMEA
Switzerland	EMEA
UK	EMEA

In [20]: `%sql select * from product_dimension limit 10;`

```
* mysql+pymysql://root:***@localhost
10 rows affected.
```

Out [20]:

product_dimension_id	productLine	productScale
----------------------	-------------	--------------

1	Classic Cars	1:10
2	Classic Cars	1:12
3	Classic Cars	1:18
4	Classic Cars	1:24
5	Motorcycles	1:10
6	Motorcycles	1:12
7	Motorcycles	1:18
8	Motorcycles	1:24
9	Motorcycles	1:32
10	Motorcycles	1:50

In [21]: `%sql select * from date_dimension limit 10;`

* mysql+pymysql://root:***@localhost
10 rows affected.

Out [21]:

orderDate	orderYear	orderQuarter	orderMonth
-----------	-----------	--------------	------------

2003-01-06	2003	1	1
2003-01-09	2003	1	1
2003-01-10	2003	1	1
2003-01-29	2003	1	1
2003-01-31	2003	1	1
2003-02-11	2003	1	2
2003-02-17	2003	1	2
2003-02-24	2003	1	2
2003-03-03	2003	1	3
2003-03-10	2003	1	3

To make querying the star schema easier and more intuitive, I also created a view that joins the **facts** table with the **dimensions** table.

In [22]: `%sql select * from facts_dimensions limit 10;`

* mysql+pymysql://root:***@localhost
10 rows affected.

Out [22]:

fact_id	location_id	product_dimension_id	orderDate	quantityOrdered	priceEach	re
1	53	1	2003-01-29	26	214.30	51
2	66	1	2003-03-24	29	197.16	5
3	63	1	2003-05-28	38	205.73	7
4	80	1	2003-07-24	37	186.44	68
5	60	1	2003-09-19	45	182.16	8
6	89	1	2003-10-20	21	212.16	44
7	5	1	2003-11-06	34	207.87	70
8	91	1	2003-11-13	23	180.01	41
9	19	1	2003-11-25	42	203.59	85
10	12	1	2003-12-05	47	203.59	95

Your first task on homework 4 is to write **DDL to created the star schema and view**. Place and execute your DDL in the following cell.

```
In [ ]: # %%sql

# create schema classicmodels_olap;

# use classicmodels_olap;

# /*
#     Your DDL goes here.
# */
```

```
In [23]: %%sql

DROP SCHEMA IF EXISTS classicmodels_olap;
CREATE SCHEMA classicmodels_olap;

USE classicmodels_olap;

CREATE TABLE location_dimension (
    location_id INT AUTO_INCREMENT,
    country VARCHAR(50) NOT NULL,
    city VARCHAR(50) NOT NULL,
    region VARCHAR(10),
```

```
PRIMARY KEY (location_id),
UNIQUE (country, city)
);

CREATE TABLE product_dimension (
    product_dimension_id INT AUTO_INCREMENT,
    productLine VARCHAR(50) NOT NULL,
    productScale VARCHAR(10) NOT NULL,
    PRIMARY KEY (product_dimension_id),
    UNIQUE (productLine, productScale)
);

CREATE TABLE date_dimension (
    orderDate DATE,
    orderYear INT NOT NULL,
    orderQuarter INT NOT NULL,
    orderMonth INT NOT NULL,
    PRIMARY KEY (orderDate)
);

CREATE TABLE facts (
    fact_id INT AUTO_INCREMENT,
    location_id INT NOT NULL,
    product_dimension_id INT NOT NULL,
    orderDate DATE NOT NULL,
    quantityOrdered INT NOT NULL,
    priceEach DECIMAL(10,2) NOT NULL,
    revenue DECIMAL(12,2) NOT NULL,
    PRIMARY KEY (fact_id),

    FOREIGN KEY (location_id)
        REFERENCES location_dimension(location_id),

    FOREIGN KEY (product_dimension_id)
        REFERENCES product_dimension(product_dimension_id),

    FOREIGN KEY (orderDate)
        REFERENCES date_dimension(orderDate)
);

CREATE OR REPLACE VIEW facts_dimensions AS
SELECT
    f.fact_id,
    f.location_id,
    f.product_dimension_id,
    f.orderDate,
    f.quantityOrdered,
    f.priceEach,
    f.revenue,
    l.country,
    l.city,
    l.region,
    d.orderYear,
    d.orderQuarter,
    d.orderMonth,
    p.productLine,
```

```

        p.productScale
FROM facts f
JOIN location_dimension l
    ON f.location_id = l.location_id
JOIN date_dimension d
    ON f.orderDate = d.orderDate
JOIN product_dimension p
    ON f.product_dimension_id = p.product_dimension_id;

```

```

* mysql+pymysql://root:***@localhost
5 rows affected.
1 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.

```

Out [23]: []

In [24]: `%sql describe classicmodels_olap.facts;`

```

* mysql+pymysql://root:***@localhost
7 rows affected.

```

Out [24]:

	Field	Type	Null	Key	Default	Extra
	fact_id	int	NO	PRI	None	auto_increment
	location_id	int	NO	MUL	None	
	product_dimension_id	int	NO	MUL	None	
	orderDate	date	NO	MUL	None	
	quantityOrdered	int	NO		None	
	priceEach	decimal(10,2)	NO		None	
	revenue	decimal(12,2)	NO		None	

In [25]: `%sql describe classicmodels_olap.location_dimension;`

```

* mysql+pymysql://root:***@localhost
4 rows affected.

```

Out [25]:

	Field	Type	Null	Key	Default	Extra
	location_id	int	NO	PRI	None	auto_increment
	country	varchar(50)	NO	MUL	None	
	city	varchar(50)	NO		None	
	region	varchar(10)	YES		None	

In [26]: `%sql describe classicmodels_olap.product_dimension;`

```

* mysql+pymysql://root:***@localhost
3 rows affected.

```

Out [26]:

	Field	Type	Null	Key	Default	Extra
	product_dimension_id	int	NO	PRI	None	auto_increment
	productLine	varchar(50)	NO	MUL	None	
	productScale	varchar(10)	NO		None	

In [27]: `%sql describe classicmodels_olap.date_dimension;`

* mysql+pymysql://root:***@localhost
4 rows affected.

Out [27]:

	Field	Type	Null	Key	Default	Extra
	orderDate	date	NO	PRI	None	
	orderYear	int	NO		None	
	orderQuarter	int	NO		None	
	orderMonth	int	NO		None	

In [28]: `%sql show tables from classicmodels_olap;`

* mysql+pymysql://root:***@localhost
5 rows affected.

Out [28]: **Tables_in_classicmodels_olap**

date_dimension
facts
facts_dimensions
location_dimension
product_dimension

Classicmodels Star Schema Loading

Your second task for the homework is to **load the schema you created from the base data in classicmodels**. This will take several SQL statements and you may create temporary, intermediate tables if you want.

Loading the data into the star schema from the `classicmodels` database requires some **joins** and **projects**. You start with `orderdetails`.

- `orderDetails` joined with `orders` can gives you the `orderDate`.
- `orderDetails` joined with `orders` joined with customers gives you `country`, `city`.
- `orderDetails` joined with `products` gives you `productLine`, `productScale`.

You basically have to write some relatively sophisticated SQL code to perform the joins, load the data and link the facts to the dimensions.

Or, ... you can load this notebook and the initial schema into something like Cursor and have it help you write the queries.

Enter your SQL for loading the schema in the cell below.

In []: # %sql

```
In [29]: %%sql

USE classicmodels_olap;

INSERT INTO location_dimension (country, city, region)
SELECT DISTINCT
    c.country,
    c.city,
    CASE
        WHEN c.country IN ('Australia', 'Hong Kong', 'Japan', 'New Zealand',
                           'Singapore') THEN 'APAC'
        WHEN c.country IN ('Canada', 'USA') THEN 'AMER'
        ELSE 'EMEA'
    END AS region
FROM classicmodels.customers c
ORDER BY c.country, c.city;

INSERT INTO product_dimension (productLine, productScale)
SELECT DISTINCT
    p.productLine,
    p.productScale
FROM classicmodels.products p
ORDER BY p.productLine, p.productScale;

INSERT INTO date_dimension (orderDate, orderYear, orderQuarter, orderMonth)
SELECT DISTINCT
    o.orderDate,
    YEAR(o.orderDate) AS orderYear,
    QUARTER(o.orderDate) AS orderQuarter,
    MONTH(o.orderDate) AS orderMonth
FROM classicmodels.orders o
ORDER BY o.orderDate;

INSERT INTO facts (
    location_id,
    product_dimension_id,
    orderDate,
    quantityOrdered,
    priceEach,
    revenue
)
SELECT
    l.location_id,
```

```

    pd.product_dimension_id,
    o.orderDate,
    od.quantityOrdered,
    od.priceEach,
    ROUND(od.quantityOrdered * od.priceEach, 2) AS revenue
FROM classicmodels.orderdetails od
JOIN classicmodels.orders o
    ON od.orderNumber = o.orderNumber
JOIN classicmodels.customers c
    ON o.customerNumber = c.customerNumber
JOIN classicmodels.products p
    ON od.productCode = p.productCode
JOIN location_dimension l
    ON c.country = l.country
    AND c.city = l.city
JOIN product_dimension pd
    ON p.productLine = pd.productLine
    AND p.productScale = pd.productScale
JOIN date_dimension d
    ON o.orderDate = d.orderDate;

```

```

* mysql+pymysql://root:***@localhost
0 rows affected.
96 rows affected.
30 rows affected.
266 rows affected.
2998 rows affected.

```

Out[29]: []

Display sample data from the tables and the view you created below.

In [31]: `%sql SELECT * FROM classicmodels_olap.location_dimension LIMIT 10;`

```

* mysql+pymysql://root:***@localhost
10 rows affected.

```

Out[31]:

location_id	country	city	region
1	Australia	Chatswood	APAC
2	Australia	Glen Waverly	APAC
3	Australia	Melbourne	APAC
4	Australia	North Sydney	APAC
5	Australia	South Brisbane	APAC
6	Austria	Graz	EMEA
7	Austria	Salzburg	EMEA
8	Belgium	Bruxelles	EMEA
9	Belgium	Charleroi	EMEA
10	Canada	Montréal	AMER

location_id	country	city	region
1	Australia	Chatswood	APAC
2	Australia	Glen Waverly	APAC
3	Australia	Melbourne	APAC
4	Australia	North Sydney	APAC
5	Australia	South Brisbane	APAC
6	Austria	Graz	EMEA
7	Austria	Salzburg	EMEA
8	Belgium	Bruxelles	EMEA
9	Belgium	Charleroi	EMEA
10	Canada	Montréal	AMER

In [32]: `%sql SELECT * FROM classicmodels_olap.product_dimension LIMIT 10;`

* mysql+pymysql://root:***@localhost
10 rows affected.

Out[32]:

product_dimension_id	productLine	productScale
1	Classic Cars	1:10
2	Classic Cars	1:12
3	Classic Cars	1:18
4	Classic Cars	1:24
5	Motorcycles	1:10
6	Motorcycles	1:12
7	Motorcycles	1:18
8	Motorcycles	1:24
9	Motorcycles	1:32
10	Motorcycles	1:50

1	Classic Cars	1:10
2	Classic Cars	1:12
3	Classic Cars	1:18
4	Classic Cars	1:24
5	Motorcycles	1:10
6	Motorcycles	1:12
7	Motorcycles	1:18
8	Motorcycles	1:24
9	Motorcycles	1:32
10	Motorcycles	1:50

In [33]: `%sql SELECT * FROM classicmodels_olap.date_dimension LIMIT 10;`

* mysql+pymysql://root:***@localhost
10 rows affected.

Out[33]:

orderDate	orderYear	orderQuarter	orderMonth
2003-01-06	2003	1	1
2003-01-09	2003	1	1
2003-01-10	2003	1	1
2003-01-29	2003	1	1
2003-01-31	2003	1	1
2003-02-11	2003	1	2
2003-02-17	2003	1	2
2003-02-24	2003	1	2
2003-03-03	2003	1	3
2003-03-10	2003	1	3

2003-01-06	2003	1	1
2003-01-09	2003	1	1
2003-01-10	2003	1	1
2003-01-29	2003	1	1
2003-01-31	2003	1	1
2003-02-11	2003	1	2
2003-02-17	2003	1	2
2003-02-24	2003	1	2
2003-03-03	2003	1	3
2003-03-10	2003	1	3

In [34]: `%sql SELECT * FROM classicmodels_olap.facts LIMIT 10;`

* mysql+pymysql://root:***@localhost
10 rows affected.

Out [34]:

fact_id	location_id	product_dimension_id	orderDate	quantityOrdered	priceEach	re
1	53	1	2003-01-29	26	214.30	51
2	66	1	2003-03-24	29	197.16	5
3	63	1	2003-05-28	38	205.73	7
4	80	1	2003-07-24	37	186.44	68
5	60	1	2003-09-19	45	182.16	8
6	89	1	2003-10-20	21	212.16	44
7	5	1	2003-11-06	34	207.87	70
8	91	1	2003-11-13	23	180.01	47
9	19	1	2003-11-25	42	203.59	85
10	12	1	2003-12-05	47	203.59	95

In [36]: `%sql SELECT * FROM classicmodels_olap.facts_dimensions LIMIT 10;`

* mysql+pymysql://root:***@localhost
10 rows affected.

Out [36]:

fact_id	location_id	product_dimension_id	orderDate	quantityOrdered	priceEach	re
---------	-------------	----------------------	-----------	-----------------	-----------	----

1	53	1	2003-01-29	26	214.30	5
2	66	1	2003-03-24	29	197.16	5
3	63	1	2003-05-28	38	205.73	7
4	80	1	2003-07-24	37	186.44	68
5	60	1	2003-09-19	45	182.16	8
6	89	1	2003-10-20	21	212.16	44
7	5	1	2003-11-06	34	207.87	70
8	91	1	2003-11-13	23	180.01	4
9	19	1	2003-11-25	42	203.59	88
10	12	1	2003-12-05	47	203.59	95

In [37]:

```
%%sql

SELECT 'location_dimension' AS table_name, COUNT(*) AS row_count
FROM classicmodels_olap.location_dimension

UNION ALL

SELECT 'product_dimension', COUNT(*)
FROM classicmodels_olap.product_dimension

UNION ALL

SELECT 'date_dimension', COUNT(*)
FROM classicmodels_olap.date_dimension

UNION ALL

SELECT 'facts', COUNT(*)
FROM classicmodels_olap.facts

UNION ALL

SELECT 'facts_dimensions', COUNT(*)
FROM classicmodels_olap.facts_dimensions;
```

* mysql+pymysql://root:***@localhost
5 rows affected.

Out [37]:

table_name	row_count
location_dimension	96
product_dimension	30
date_dimension	266
facts	2998
facts_dimensions	2998

In [38]: `%sql SELECT region, COUNT(*) FROM classicmodels_olap.location_dimension GROUP BY region`
 * mysql+pymysql://root:***@localhost
 3 rows affected.

Out [38]:

region	COUNT(*)
APAC	13
EMEA	57
AMER	26

Classicmodels Star Schema Queries

Some Examples

The view makes the queries a little easier to understand.

Normally, I would load this notebook into Cursor and use it to generate some visualizations.

But, you have probably noticed that I am both lazy and very busy.

In [39]: `%%sql`

```

/*
    Take a slice for 2004.
*/
use classicmodels_olap;

select
    orderYear, region, productLine, count(*) as noOfPurchases,
    round(sum(revenue),2) as totalRevenue
from
    facts_dimensions
where
    orderYear=2004
group by productLine, region
order by region, productLine;
```

```
* mysql+pymysql://root:***@localhost
```

```
0 rows affected.
```

```
21 rows affected.
```

```
Out[39]:
```

orderYear	region	productLine	noOfPurchases	totalRevenue
2004	AMER	Classic Cars	147	554092.09
2004	AMER	Motorcycles	82	261143.02
2004	AMER	Planes	62	175223.56
2004	AMER	Ships	48	127889.01
2004	AMER	Trains	9	20753.90
2004	AMER	Trucks and Buses	75	232586.47
2004	AMER	Vintage Cars	97	278215.63
2004	APAC	Classic Cars	59	222643.62
2004	APAC	Motorcycles	28	92773.01
2004	APAC	Planes	37	101875.93
2004	APAC	Ships	15	40525.89
2004	APAC	Trains	4	8113.38
2004	APAC	Trucks and Buses	25	87630.42
2004	APAC	Vintage Cars	52	141195.22
2004	EMEA	Classic Cars	256	986401.02
2004	EMEA	Motorcycles	61	173327.81
2004	EMEA	Planes	69	194871.97
2004	EMEA	Ships	63	168911.20
2004	EMEA	Trains	28	67418.25
2004	EMEA	Trucks and Buses	43	145173.11
2004	EMEA	Vintage Cars	161	435141.00

```
In [40]: %%sql

/*
    Drill down to include country.
*/
use classicmodels_olap;

select
    orderYear, region, country, productLine, count(*) as noOfPurchases,
    round(sum(revenue),2) as totalRevenue
from
    facts_dimensions
where
    orderYear=2004
```

```
group by productLine, region, country  
order by region, country, productLine;
```

```
* mysql+pymysql://root:***@localhost  
0 rows affected.  
105 rows affected.
```

Out [40]:	orderYear	region	country	productLine	noOfPurchases	totalRevenue
	2004	AMER	Canada	Classic Cars	5	18576.32
	2004	AMER	Canada	Motorcycles	1	3726.90
	2004	AMER	Canada	Planes	10	23540.47
	2004	AMER	Canada	Ships	14	36605.16
	2004	AMER	Canada	Trucks and Buses	8	21440.18
	2004	AMER	Canada	Vintage Cars	8	19515.00
	2004	AMER	USA	Classic Cars	142	535515.77
	2004	AMER	USA	Motorcycles	81	257416.12
	2004	AMER	USA	Planes	52	151683.09
	2004	AMER	USA	Ships	34	91283.85
	2004	AMER	USA	Trains	9	20753.90
	2004	AMER	USA	Trucks and Buses	67	211146.29
	2004	AMER	USA	Vintage Cars	89	258700.63
	2004	APAC	Australia	Classic Cars	17	69649.34
	2004	APAC	Australia	Motorcycles	10	33024.95
	2004	APAC	Australia	Planes	14	34059.82
	2004	APAC	Australia	Ships	1	2063.76
	2004	APAC	Australia	Trucks and Buses	9	32362.30
	2004	APAC	Australia	Vintage Cars	14	33053.01
	2004	APAC	Japan	Classic Cars	5	24790.02
	2004	APAC	Japan	Motorcycles	9	32642.56
	2004	APAC	Japan	Planes	16	41534.63
	2004	APAC	Japan	Ships	3	9060.98
	2004	APAC	Japan	Trucks and Buses	3	11385.51
	2004	APAC	Japan	Vintage Cars	6	13913.07
	2004	APAC	New Zealand	Classic Cars	19	67621.45
	2004	APAC	New Zealand	Motorcycles	7	23036.56
	2004	APAC	New Zealand	Planes	7	26281.48
	2004	APAC	New Zealand	Ships	11	29401.15
	2004	APAC	New Zealand	Trains	4	8113.38
	2004	APAC	New Zealand	Vintage Cars	26	78908.25
	2004	APAC	Philippines	Classic Cars	2	11753.90

orderYear	region	country	productLine	noOfPurchases	totalRevenue
2004	APAC	Philippines	Motorcycles	2	4068.94
2004	APAC	Singapore	Classic Cars	16	48828.91
2004	APAC	Singapore	Trucks and Buses	13	43882.61
2004	APAC	Singapore	Vintage Cars	6	15320.89
2004	EMEA	Austria	Classic Cars	4	22380.74
2004	EMEA	Austria	Motorcycles	5	18039.49
2004	EMEA	Austria	Planes	1	2393.60
2004	EMEA	Austria	Vintage Cars	2	6419.84
2004	EMEA	Belgium	Classic Cars	1	4080.00
2004	EMEA	Belgium	Planes	1	5624.79
2004	EMEA	Belgium	Ships	11	27855.06
2004	EMEA	Belgium	Trains	2	6187.56
2004	EMEA	Belgium	Vintage Cars	8	30587.96
2004	EMEA	Denmark	Classic Cars	16	63981.72
2004	EMEA	Denmark	Planes	2	7208.80
2004	EMEA	Denmark	Ships	7	15268.51
2004	EMEA	Denmark	Trains	3	5907.12
2004	EMEA	Denmark	Trucks and Buses	2	7560.55
2004	EMEA	Denmark	Vintage Cars	3	7305.93
2004	EMEA	Finland	Classic Cars	11	47699.02
2004	EMEA	Finland	Motorcycles	9	26386.84
2004	EMEA	Finland	Trains	2	4862.35
2004	EMEA	France	Classic Cars	49	191252.70
2004	EMEA	France	Motorcycles	31	84712.49
2004	EMEA	France	Planes	9	24914.19
2004	EMEA	France	Ships	12	35809.73
2004	EMEA	France	Trains	6	12887.48
2004	EMEA	France	Trucks and Buses	8	29504.46
2004	EMEA	France	Vintage Cars	48	127578.96
2004	EMEA	Germany	Classic Cars	20	82576.64
2004	EMEA	Germany	Motorcycles	2	3870.84
2004	EMEA	Germany	Planes	8	19600.79

orderYear	region	country	productLine	noOfPurchases	totalRevenue
2004	EMEA	Germany	Ships	2	4510.02
2004	EMEA	Germany	Trains	2	5184.01
2004	EMEA	Germany	Trucks and Buses	2	7976.22
2004	EMEA	Germany	Vintage Cars	5	10113.18
2004	EMEA	Ireland	Classic Cars	6	26413.74
2004	EMEA	Ireland	Motorcycles	2	4553.96
2004	EMEA	Ireland	Planes	4	11033.45
2004	EMEA	Ireland	Trains	2	2698.80
2004	EMEA	Ireland	Trucks and Buses	1	3426.20
2004	EMEA	Ireland	Vintage Cars	1	1772.12
2004	EMEA	Italy	Classic Cars	20	83225.07
2004	EMEA	Italy	Planes	9	27069.60
2004	EMEA	Italy	Ships	4	9331.55
2004	EMEA	Italy	Trains	1	2018.72
2004	EMEA	Italy	Trucks and Buses	2	4782.12
2004	EMEA	Italy	Vintage Cars	21	52681.34
2004	EMEA	Norway	Classic Cars	4	22392.46
2004	EMEA	Norway	Motorcycles	6	17876.32
2004	EMEA	Norway	Trains	2	4022.96
2004	EMEA	Norway	Trucks and Buses	2	5468.96
2004	EMEA	Norway	Vintage Cars	1	2753.76
2004	EMEA	Norway	Planes	10	25719.05
2004	EMEA	Norway	Vintage Cars	7	19079.12
2004	EMEA	Spain	Classic Cars	56	207585.92
2004	EMEA	Spain	Motorcycles	6	17887.87
2004	EMEA	Spain	Planes	13	38630.53
2004	EMEA	Spain	Ships	10	26044.04
2004	EMEA	Spain	Trains	3	8047.51
2004	EMEA	Spain	Trucks and Buses	15	45621.03
2004	EMEA	Spain	Vintage Cars	34	96064.94
2004	EMEA	Sweden	Classic Cars	10	37586.88
2004	EMEA	Sweden	Ships	2	7303.76

orderYear	region	country	productLine	noOfPurchases	totalRevenue
2004	EMEA	Sweden	Trains	1	3033.28
2004	EMEA	Sweden	Trucks and Buses	11	40833.57
2004	EMEA	Sweden	Vintage Cars	7	19072.29
2004	EMEA	Switzerland	Classic Cars	31	108777.92
2004	EMEA	UK	Classic Cars	28	88448.21
2004	EMEA	UK	Planes	12	32677.17
2004	EMEA	UK	Ships	15	42788.53
2004	EMEA	UK	Trains	4	12568.46
2004	EMEA	UK	Vintage Cars	24	61711.56

Your Queries

Load your schema into ChatGPT, Cursor, etc. and come up with 5 "interesting" OLAP like queries. Execute and explain your queries below.

Query 1: Total revenue by year and region

Explanation:

- This query performs a roll-up over the fact table by combining the time dimension and the location dimension. Specifically, it groups the data by `orderYear` and `region`, and then computes two measures: the number of purchase line items and the total revenue. Since each row in the fact table represents an order detail line, `COUNT(*)` measures the number of purchased line items rather than the number of distinct orders.
- This query is useful because it shows how revenue changes over time across major geographic regions. By comparing `APAC`, `AMER`, and `EMEA` for each year, we can identify which regions are the strongest contributors to revenue and whether regional performance changes from year to year.

In [41]: `%%sql`

```
USE classicmodels_olap;

SELECT
    orderYear,
    region,
    COUNT(*) AS noOfPurchases,
    ROUND(SUM(revenue), 2) AS totalRevenue
FROM facts_dimensions
```

```
GROUP BY orderYear, region
ORDER BY orderYear, totalRevenue DESC;
```

```
* mysql+pymysql://root:***@localhost
0 rows affected.
10 rows affected.
```

Out [41]:

orderYear	region	noOfPurchases	totalRevenue
2003	EMEA	484	1519511.84
2003	AMER	382	1225910.04
2003	APAC	186	572198.51
2004	EMEA	681	2171244.36
2004	AMER	520	1649903.68
2004	APAC	220	694757.47
2005	EMEA	250	829956.08
2005	AMER	172	603650.19
2005	APAC	101	337330.44
2026	EMEA	2	292.86

Query 2: Product line revenue ranking

Explanation:

- This query analyzes the product dimension by grouping sales according to `productLine`. It computes the number of purchase line items, the total quantity ordered, and the total revenue for each product line. This is a roll-up over the product hierarchy because it summarizes many individual order detail records into broader product categories.
- The query is useful for identifying which product categories contribute the most to the business. Comparing `totalQuantityOrdered` with `totalRevenue` can also reveal whether a product line sells many units at lower prices or fewer units at higher revenue. This makes the query helpful for understanding both demand volume and revenue contribution.

In [42]:

```
%%sql

USE classicmodels_olap;

SELECT
    productLine,
    COUNT(*) AS noOfPurchases,
    SUM(quantityOrdered) AS totalQuantityOrdered,
    ROUND(SUM(revenue), 2) AS totalRevenue
FROM facts_dimensions
```

```
GROUP BY productLine
ORDER BY totalRevenue DESC;
```

```
* mysql+pymysql://root:***@localhost
0 rows affected.
7 rows affected.
```

Out [42]:

productLine	noOfPurchases	totalQuantityOrdered	totalRevenue
Classic Cars	1010	35582	3853922.49
Vintage Cars	657	22935	1797831.63
Motorcycles	361	12784	1121718.98
Trucks and Buses	308	11001	1024113.57
Planes	336	11872	954637.54
Ships	245	8532	663998.34
Trains	81	2818	188532.92

Query 3: Quarterly revenue trend

Explanation:

- This query uses the date dimension to aggregate revenue by `orderYear` and `orderQuarter`. This is an OLAP-style time-series roll-up because individual transaction-level facts are summarized into quarterly periods. The query calculates both the number of purchase line items and the total revenue for each quarter.
- This query is useful for detecting seasonal sales patterns. For example, if one quarter consistently has higher revenue, that may suggest seasonal demand, promotional effects, or changes in customer purchasing behavior. It also allows us to compare performance within the same year rather than only looking at annual totals.

In [43]:

```
%%sql

USE classicmodels_olap;

SELECT
    orderYear,
    orderQuarter,
    COUNT(*) AS noOfPurchases,
    ROUND(SUM(revenue), 2) AS totalRevenue
FROM facts_dimensions
GROUP BY orderYear, orderQuarter
ORDER BY orderYear, orderQuarter;
```

```
* mysql+pymysql://root:***@localhost
0 rows affected.
11 rows affected.
```

Out [43]:

orderYear	orderQuarter	noOfPurchases	totalRevenue
2003	1	130	405885.55
2003	2	163	515754.91
2003	3	197	616895.32
2003	4	562	1779084.61
2004	1	244	799579.31
2004	2	246	779271.81
2004	3	330	1028690.38
2004	4	601	1908364.01
2005	1	313	984641.15
2005	2	210	786295.56
2026	2	2	292.86

Query 4: Country-level drill down within each region

Explanation:

- This query performs a drill-down on the location dimension. Instead of stopping at the regional level, it breaks each region into individual countries and computes the number of purchase line items and total revenue for each country. This gives a more detailed view of geographic performance.
- The query is useful because a region-level summary can hide important differences between countries. For example, a region may have strong total revenue because of one or two dominant countries. By drilling down to the country level, we can identify the specific markets that drive revenue within each region.

In [44]:

```
%%sql
USE classicmodels_olap;

SELECT
    region,
    country,
    COUNT(*) AS noOfPurchases,
    ROUND(SUM(revenue), 2) AS totalRevenue
FROM facts_dimensions
GROUP BY region, country
ORDER BY region, totalRevenue DESC;
```

```
* mysql+pymysql://root:***@localhost
0 rows affected.
22 rows affected.
```

Out [44]:

region	country	noOfPurchases	totalRevenue
AMER	USA	1004	3273552.05
AMER	Canada	70	205911.86
APAC	Australia	185	562582.59
APAC	New Zealand	149	476847.01
APAC	Singapore	79	263997.78
APAC	Japan	52	167909.95
APAC	Philippines	26	87468.30
APAC	Hong Kong	16	45480.79
EMEA	Spain	342	1099389.09
EMEA	France	316	1007666.88
EMEA	UK	144	436947.44
EMEA	Italy	121	360616.81
EMEA	Finland	92	295149.35
EMEA	Denmark	63	218994.92
EMEA	Germany	62	196470.99
EMEA	Austria	55	188540.06
EMEA	Sweden	57	187638.35
EMEA	Norway	53	166621.51
EMEA	Switzerland	31	108777.92
EMEA	Norway	32	104224.79
EMEA	Belgium	33	100068.76
EMEA	Ireland	16	49898.27

Query 5: Product line performance by region

- This query combines the location dimension and the product dimension by grouping the data by `region` and `productLine`. It measures both total quantity ordered and total revenue for each product line within each geographic region. This is an OLAP-style multidimensional aggregation because it analyzes sales across two dimensions at the same time.
- The query is useful for comparing product preferences across regions. A product line that performs well in one region may not be the strongest category in another region. This type of query can support regional sales strategy, inventory planning, and marketing decisions by showing which product categories are most important in each market.

In [45]: `%%sql`

```
USE classicmodels_olap;

SELECT
    region,
    productLine,
    SUM(quantityOrdered) AS totalQuantityOrdered,
    ROUND(SUM(revenue), 2) AS totalRevenue
FROM facts_dimensions
GROUP BY region, productLine
ORDER BY region, totalRevenue DESC;
```

```
* mysql+pymysql://root:***@localhost
0 rows affected.
21 rows affected.
```

Out [45]:

region	productLine	totalQuantityOrdered	totalRevenue
AMER	Classic Cars	12111	1322606.40
AMER	Vintage Cars	8757	689267.95
AMER	Motorcycles	5191	470817.63
AMER	Trucks and Buses	4449	411823.04
AMER	Planes	3805	299085.58
AMER	Ships	2914	226269.48
AMER	Trains	912	59593.83
APAC	Classic Cars	5179	551274.87
APAC	Vintage Cars	4518	349750.94
APAC	Motorcycles	2481	221386.72
APAC	Planes	2554	208347.10
APAC	Trucks and Buses	1897	184129.66
APAC	Ships	810	64316.85
APAC	Trains	362	25080.28
EMEA	Classic Cars	18292	1980041.22
EMEA	Vintage Cars	9660	758812.74
EMEA	Planes	5513	447204.86
EMEA	Motorcycles	5112	429514.63
EMEA	Trucks and Buses	4655	428160.87
EMEA	Ships	4808	373412.01
EMEA	Trains	1544	103858.81